
NEI OS — Full System Infrastructure Blueprint

1. What NEI Is Technically

Neural Eye Intelligence (NEI) is a multi-tenant, AI-enabled ophthalmology healthcare platform.

It connects:

PATIENT

|



PATIENT APP

|



NEI CORE PLATFORM

|

└— Identity & Access

└— Patient Health Record

└— Clinical Management

└— Appointment Engine

└— Telemedicine

└— Prescription Engine

└— Messaging

└— Notification Engine

└— AI / EyeGPT

└— Imaging & Documents

└— Analytics

└— Integration Layer



The system should be designed so that additional modules can be added without rebuilding the platform.

2. NEI Ecosystem

NEI should consist of these major products:

NEI OS

The central platform.

NEI Patient

Mobile application for patients.

NEI Doctor

Clinical workspace for ophthalmologists.

NEI Clinic

Administrative and business management.

NEI Connect

Telemedicine and communication infrastructure.

NEI Script

Digital prescription infrastructure.

NEI Optics

Optical-shop ecosystem.

NEI Pharmacy

Pharmacy prescription and medication infrastructure.

EyeGPT

AI intelligence layer.

NEI Analytics

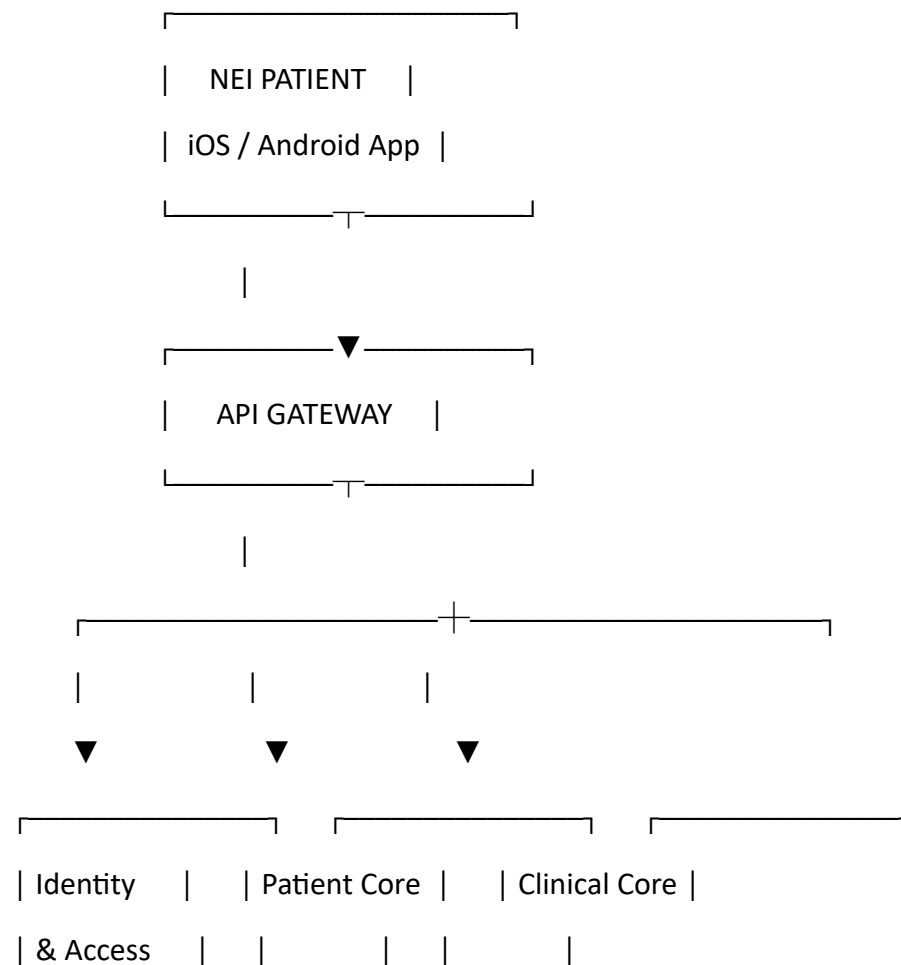
Business and clinical analytics.

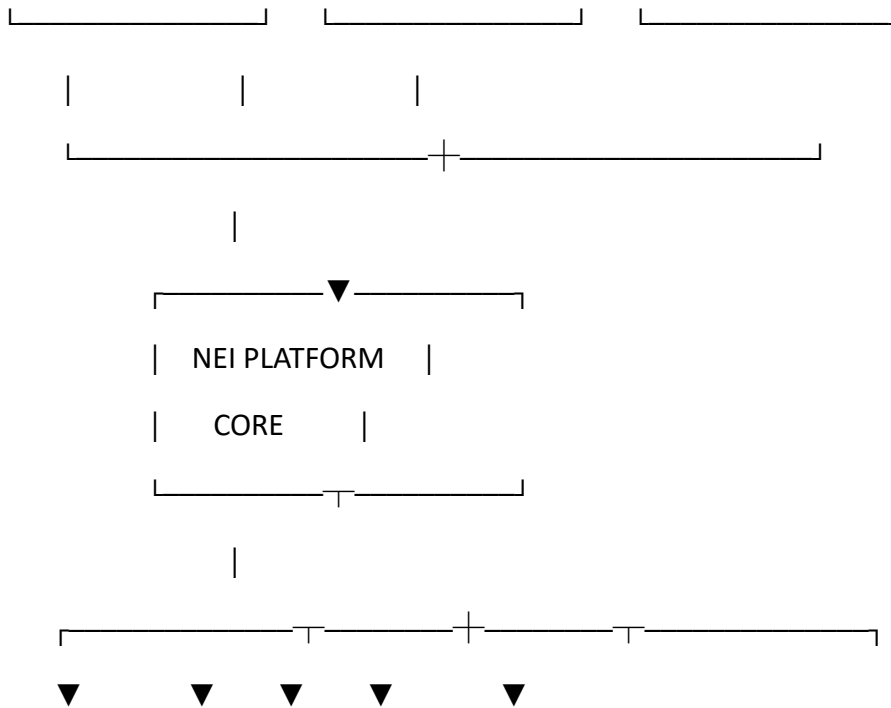
NEI XR

Future immersive diagnostics and rehabilitation.

3. Core Architecture

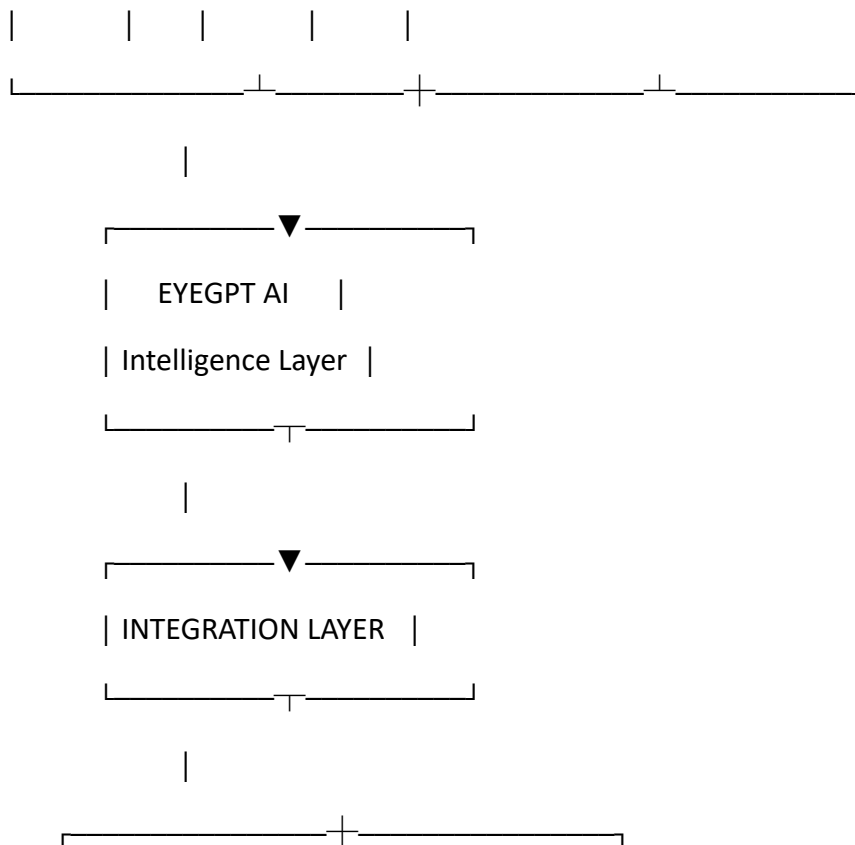
I recommend this architecture:





Appointment Telemed Prescription Messaging Documents

Engine Engine Engine Engine Engine





4. Frontend Infrastructure

A. Patient Mobile App

Use:

Flutter

Platforms:

- iOS
- Android

Main sections:

Patient App

|

|— Authentication

|— Home

|— Appointments

|— Doctors

|— Telemedicine

|— Eye Health

|— Prescriptions

|— Medications

|— Documents

|— Vision History

|— Messages

- └— EyeGPT
- └— Optical
- └— Notifications
- └ Profile & Settings

5. Patient Home

The patient should see only the information that matters.

Good Morning, Sarah

Your Eye Care Today

Upcoming Appointment

Dr. Ahmad

Today • 3:30 PM

[Join Consultation]

Medication Reminder

Eye Drops

1 drop • 3 times daily

[Mark as Taken]

| Your Eye Health |
| Last examination: Aug 2026 |
| [View History] |
|_____|

Ask EyeGPT

The dashboard should dynamically prioritize what is relevant.

6. Patient Authentication

The authentication infrastructure should support:

Sign Up

- phone number + OTP
- email
- Google
- Apple

Sign In

- phone OTP
- email/password
- Google
- Apple

Security

- MFA
- session management
- refresh tokens
- device management
- suspicious-login detection

- account recovery
-

7. Patient Identity

Every patient receives a unique:

NEI Patient ID

Example internally:

PAT-xxxxxxx

This ID becomes the anchor for all patient information.

The same patient can then connect to:

- multiple doctors,
 - multiple clinics,
 - pharmacies,
 - optical shops,
 - telemedicine consultations.
-

8. Patient Health Record

This is one of the most important parts of NEI.

Create a longitudinal:

Eye Health Record

It should contain:

Patient

|

└— Personal Information

└— Allergies

└— Medical Conditions

└— Eye Conditions

- └— Vision History
 - └— Prescriptions
 - └— Medications
 - └— Procedures
 - └— Surgeries
 - └— Clinical Visits
 - └— Diagnostic Tests
 - └— Imaging
 - └— Doctor Notes
 - └— Reports
 - └— Care Timeline
-

9. Vision History

Instead of showing medical information as a giant table, create a timeline.

2026

|

└— Aug 20

| Eye Examination

| Dr. Ahmad

|

└— Jul 14

| New Prescription

|

└— May 02

| OCT Examination

|

└─ Jan 15

Follow-up

The patient sees a simple version.

The doctor sees the detailed clinical version.

10. Doctor Platform

The doctor application should be a web application.

Recommended:

Next.js + TypeScript + Tailwind

Dashboard:

Doctor Dashboard

|

└─ Today

└─ Appointments

└─ Waiting Room

└─ Patients

└─ Clinical Records

└─ Prescriptions

└─ Imaging

└─ Telemedicine

└─ Messages

└─ EyeGPT

└─ Tasks

└─ Reports

└─ Settings

11. Doctor's Patient Workspace

When the doctor opens a patient:

Patient

|

└─ Overview

└─ Medical History

└─ Eye History

└─ Previous Visits

└─ Current Problems

└─ Medications

└─ Prescriptions

└─ Imaging

└─ Documents

└─ Clinical Notes

└─ Appointments

└─ Messages

└─ AI Summary

The doctor should be able to understand the patient in seconds.

12. Clinical Encounter

Every consultation becomes a structured clinical encounter.

Encounter

|

- |— Patient
- |— Doctor
- |— Date/time
- |— Reason for visit
- |— Symptoms
- |— Examination
- |— Findings
- |— Diagnosis
- |— Treatment
- |— Prescription
- |— Clinical Notes
- |— Attachments
- |— Follow-up
- └— AI Summary

This is the foundation of the clinical system.

13. Ophthalmology-Specific Clinical Data

NEI should not be a generic EHR.

It should understand ophthalmology.

Potential structured data:

Visual Acuity

Right Eye

Left Eye

Both Eyes

Refraction

Sphere

Cylinder

Axis

Add

PD

Intraocular Pressure

Right Eye

Left Eye

Method

Time

Eye Examination

Cornea

Conjunctiva

Anterior Chamber

Lens

Retina

Optic Nerve

Macula

Vitreous

Diagnostic Testing

OCT

Visual Field

Fundus Photography

Topography

Pachymetry

Ocular Biometry

This specialty-specific structure is one of NEI's strongest potential advantages.

14. Appointment Engine

Central appointment service:

Appointment Engine

|

├— Doctor availability

├— Clinic schedules

├— Multiple branches

├— Appointment types

├— Duration

├— Online consultation

├— Physical consultation

├— Rescheduling

├— Cancellation

├— Waiting list

├— Reminders

└— No-show tracking

Appointment statuses:

REQUESTED

CONFIRMED

CHECKED_IN

IN_PROGRESS

COMPLETED

CANCELLED

NO_SHOW

15. Telemedicine — NEI Connect

Architecture:

Patient

|



Waiting Room

|



Session Authentication

|



WebRTC

|

└— Video

└— Audio

└— Chat

└— File Sharing

└— Screen Sharing

|



Consultation Record

|



EyeGPT Summary

|



Prescription / Follow-up

Use a proven video infrastructure rather than building the entire WebRTC stack from scratch initially.

16. NEI Script

Digital prescription service.

Doctor

|



Create Prescription

|



Clinical Validation

|



Digital Signature

|



Prescription ID

|

└─ QR Code

└─ PDF

└─ Digital Record

Patient receives it automatically.

Pharmacy can verify it.

17. Prescription Lifecycle

DRAFT



ISSUED



SENT



VIEWED



DISPENSED



PARTIALLY_DISPENSED



COMPLETED

This creates traceability.

18. Pharmacy Infrastructure

Pharmacy portal:

Pharmacy



├— Dashboard

├— Prescription Verification

├— Patients

├— Medications

└─ Dispensing

└─ Refill Requests

└─ Inventory

└─ Reports

Pharmacist scans:

NEI Prescription QR

The system verifies:

- authenticity,
- patient,
- doctor,
- prescription status,
- expiration.

19. NEI Optics

Optical shops receive prescriptions through NEI.

Clinic

|



Prescription

|



Patient

|



NEI Optics

|

└─ Frames

└─ Lenses

└─ Orders

└─ Inventory

└─ Delivery

Future:

- face analysis,
- AI recommendations,
- AR try-on.

These should come after the core clinical infrastructure.

20. Messaging Infrastructure

NEI needs centralized communication.

Patient

|

└─ Doctor

└─ Clinic

└─ Pharmacy

└─ Optical Shop

But permissions must control who can communicate with whom.

Messages should support:

- text,
- images,
- documents,
- prescriptions,
- appointment links.

21. Notification Engine

One centralized notification service.

Channels:

Push

SMS

Email

In-App

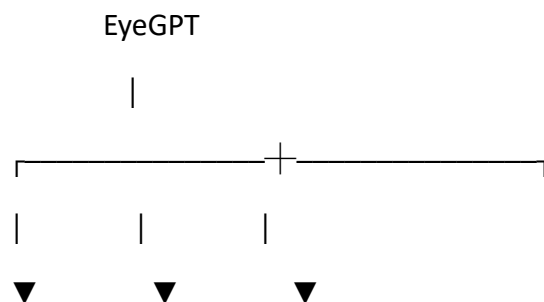
Events:

- appointment confirmation,
- appointment reminder,
- medication reminder,
- prescription issued,
- prescription ready,
- doctor message,
- telemedicine reminder,
- follow-up reminder.

22. EyeGPT Infrastructure

EyeGPT should NOT be a single chatbot.

It should be an AI layer integrated throughout NEI.



Patient AI	Doctor AI	Business AI
▼	▼	▼
Education	SOAP Notes	Analytics
Guidance	Summaries	Predictions
Reminders	Documentation	Automation

23. EyeGPT Patient

The patient can ask:

“What does my prescription mean?”

“When is my next appointment?”

“Remind me about my eye drops.”

“What should I prepare before my appointment?”

EyeGPT should access only information that the patient is authorized to see.

24. EyeGPT Doctor

Doctor AI can:

- summarize patient history,
- summarize previous consultations,
- generate draft SOAP notes,
- summarize uploaded reports,
- structure clinical information,
- prepare follow-up reminders.

Important:

AI generates assistance. The doctor remains responsible for clinical decisions.

25. AI Architecture

User

|



AI Gateway

|



Permission Check

|



Context Builder

|

├— Patient data

├— Clinical data

├— Conversation

└— Relevant documents

|



RAG / Knowledge Layer

|



LLM

|



Safety / Validation Layer

|



Response

Never allow the AI to blindly query the entire database.

26. AI Memory

EyeGPT should have different memory levels.

Session Memory

Current conversation.

Patient Context

Authorized patient information.

Clinical Context

Relevant medical record.

Knowledge Base

Approved ophthalmology information.

This separation is critical.

27. Document Infrastructure

NEI will eventually handle:

- medical reports,
- prescriptions,
- PDFs,
- OCT reports,
- imaging,
- referral letters,
- insurance documents.

Use object storage such as:

AWS S3

with encrypted storage.

Database stores metadata, not large files.

28. Imaging Infrastructure

Future architecture:

Diagnostic Device

|



Image Upload

|



Secure Object Storage

|



Imaging Service

|

├— Metadata

├— Viewer

└— AI Analysis

|



Clinical Record

For serious medical imaging, design for **DICOM compatibility** from the beginning even if advanced imaging is introduced later.

29. Core Database

Primary database:

PostgreSQL

Core entities:

users

patients

doctors

clinics

organizations

branches

roles

permissions

appointments

encounters

clinical_notes

diagnoses

symptoms

procedures

prescriptions

prescription_items

medications

pharmacies

dispensing_records

eye_examinations
visual_acuity
refractions
iop_measurements
diagnostic_tests
imaging_studies

documents
messages
notifications

ai_sessions
ai_messages
ai_summaries

optical_products
optical_orders
inventory

audit_logs

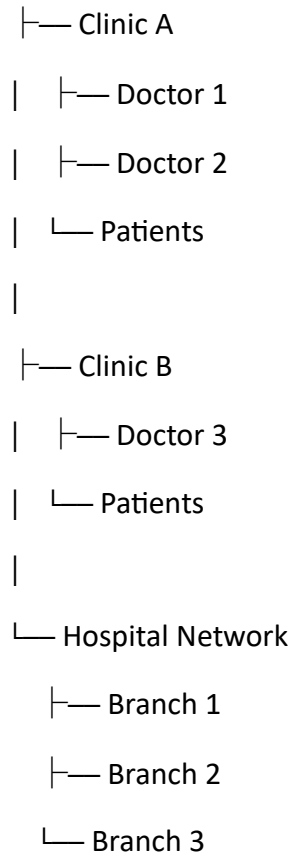
30. Multi-Tenant Architecture

This is extremely important.

NEI should support:

NEI

|



Every organization's data must be isolated.

A clinic must never be able to access another clinic's data unless explicit patient-sharing/authorization rules allow it.

31. Roles & Permissions

Create RBAC.

Patient

Can access:

- own records,
- appointments,
- prescriptions,
- messages.

Doctor

Can access:

- assigned/authorized patients,
- clinical records,
- consultations,
- prescriptions.

Nurse

Limited clinical access.

Receptionist

Appointments and patient administration.

Clinic Admin

Clinic-level management.

Pharmacy

Prescription information authorized for dispensing.

Optical Shop

Prescription and order information authorized for optical services.

NEI Super Admin

Platform administration.

32. Security Architecture

Security should be built into the infrastructure, not added later.

User

|



Authentication

|



Authorization

|



Tenant Validation

|



Permission Check

|



API

|



Encrypted Data

Implement:

- TLS,
 - encryption at rest,
 - MFA,
 - RBAC,
 - audit logs,
 - session management,
 - rate limiting,
 - API security,
 - secrets management,
 - backup,
 - disaster recovery.
-

33. Audit System

Every important action should be logged.

Example:

Doctor Ahmed

Viewed Patient X

09:42

or:

Doctor Ahmed

Issued Prescription

10:15

or:

Admin

Changed User Permission

11:03

Audit logs should be immutable and retained according to applicable regulatory requirements.

34. API Infrastructure

Use REST APIs initially.

Possible structure:

/api/v1/auth

/api/v1/patients

/api/v1/doctors

/api/v1/clinics

/api/v1/appointments

/api/v1/encounters

/api/v1/prescriptions

/api/v1/medications

/api/v1/telemedicine

/api/v1/messages

/api/v1/documents

/api/v1/imaging

/api/v1/optics

/api/v1/pharmacy

/api/v1/ai

/api/v1/notifications

/api/v1/analytics

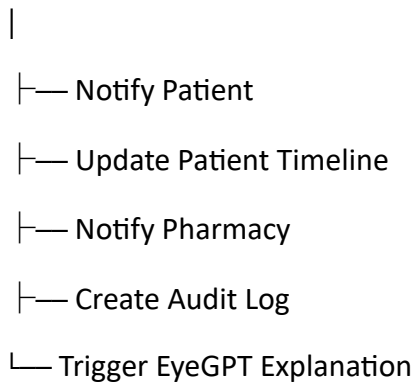
Later, selected services can move to event-driven architecture.

35. Event System

NEI should eventually have an event bus.

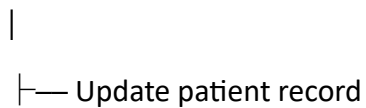
Example:

PrescriptionCreated



Another:

AppointmentCompleted



- └─ Trigger follow-up
- └─ Generate AI summary
- └─ Notify patient
- └─ Update analytics

This makes the ecosystem scalable.

36. Recommended Backend Structure

For MVP:

Backend

FastAPI + PostgreSQL + Redis

Use a modular monolith:

NEI Backend

|

- └─ Auth Module
- └─ Patient Module
- └─ Clinical Module
- └─ Appointment Module
- └─ Prescription Module
- └─ Telemedicine Module
- └─ Messaging Module
- └─ Notification Module
- └─ Document Module
- └─ AI Module
- └─ Analytics Module
- └─ Integration Module

Do NOT start with 20 microservices.

When scale demands it, extract individual services.

37. Redis

Use Redis for:

- caching,
 - sessions,
 - OTP,
 - rate limiting,
 - queues,
 - temporary state,
 - real-time features.
-

38. Background Jobs

Use a job queue for tasks that should not block the user.

Examples:

Generate AI summary

Send email

Send SMS

Process document

Generate prescription PDF

Process imaging

Generate analytics

Send reminders

39. Real-Time Infrastructure

Use WebSockets where appropriate.

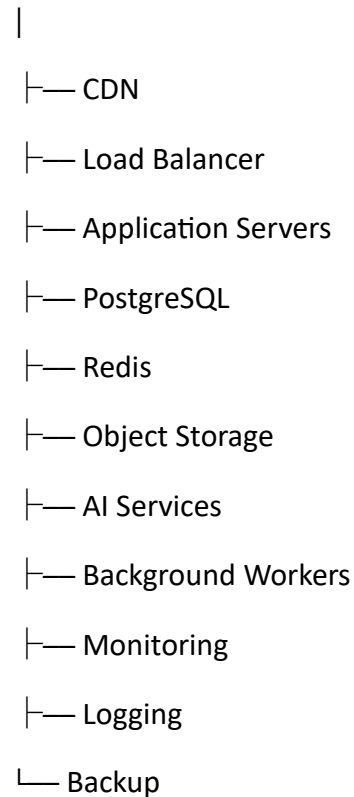
Examples:

- telemedicine waiting room,
- doctor-patient chat,
- appointment status,
- notifications,
- clinic queue.

40. Cloud Infrastructure

A strong initial architecture:

Cloud



AWS or Azure are both reasonable.

For healthcare deployments, select the actual services and regions based on the jurisdictions where NEI operates and their healthcare compliance requirements.

41. DevOps

Use:

- Docker
- GitHub/GitLab
- CI/CD
- infrastructure as code
- automated testing
- staging environment
- production environment

Environments:

LOCAL



DEVELOPMENT



STAGING



PRODUCTION

Never develop directly against production.

42. Monitoring

NEI needs centralized monitoring.

Track:

Infrastructure

- CPU
- memory

- storage
- network

Application

- API latency
- errors
- failed requests
- database performance

Healthcare

- failed prescriptions
- failed appointment notifications
- telemedicine failures

AI

- latency
- token usage
- failures
- hallucination/safety signals
- human overrides

43. Backup & Disaster Recovery

Critical data requires:

- automated database backups,
- encrypted backups,
- point-in-time recovery,
- geographically appropriate redundancy,
- disaster recovery procedures.

Define:

RPO — Recovery Point Objective

and

RTO — Recovery Time Objective

before production launch.

44. Integration Layer

NEI should never hard-code every external provider into the core.

Create:

NEI Integration Hub

NEI

|



Integration Hub

|

├— Pharmacy APIs

├— Payment APIs

├— SMS

├— Email

├— Video

├— EHR/HIS

├— Insurance

├— Optical

├— Medical Devices

└— AI Providers

This makes providers replaceable.

45. Healthcare Interoperability

Design the system with interoperability in mind.

Potential standards:

- HL7
- FHIR
- DICOM
- OAuth2
- OpenID Connect

FHIR should become particularly important for exchanging patient/clinical information with external healthcare systems.

46. Analytics Infrastructure

Three levels:

Patient Analytics

- appointment adherence,
- medication adherence,
- follow-ups.

Doctor Analytics

- consultations,
- patient volume,
- appointment performance.

Clinic Analytics

- revenue,
- utilization,
- patient growth,
- no-show rate.

Analytics must respect privacy and access permissions.

47. Admin Platform

NEI itself needs an internal administration system.

NEI Admin

|

├— Organizations

├— Clinics

├— Users

├— Doctors

├— Patients

├— Subscriptions

├— Payments

├— Integrations

├— Support

├— Security

├— Audit Logs

├— AI Usage

└— System Health

48. Subscription Infrastructure

For SaaS:

Organization

|



Subscription

|

└─ Plan

└─ Users

└─ Features

└─ Usage

└─ Billing

└─ Renewal

Plans could eventually be:

Starter

Small clinic.

Professional

Growing clinic.

Enterprise

Hospitals and healthcare groups.

49. Payment Architecture

Do not connect payment logic directly into every module.

Create:

Payment Service

|

└─ Subscription

└─ Consultation Fees

└─ Marketplace Orders

└─ Refunds

└— Invoices

└— Transactions

The actual payment providers can vary by country.

50. Notification Architecture

Centralized:

Event

|



Notification Service

|

└— Push

└— SMS

└— Email

└— In-App

The user should control notification preferences.

51. Search Infrastructure

Eventually use a search engine for:

- patients,
- doctors,
- clinics,
- appointments,
- documents.

But do not expose sensitive information through unrestricted search.

Search must always run through authorization rules.

52. AI Knowledge Infrastructure

Create an approved ophthalmology knowledge base.

Medical Sources

|



Content Validation

|



Knowledge Processing

|



Vector Database

|



RAG

|



EyeGPT

The knowledge base should be versioned.

53. AI Safety Architecture

This deserves its own layer.

User Question

|



Intent Detection

|



Permission Check

|



Clinical Risk Detection

|

|— Low Risk → Normal AI

|

|— Moderate → Educational response + doctor recommendation

|

|— High Risk → Escalation / urgent care guidance

EyeGPT should not present itself as a doctor.

54. Emergency Escalation

For certain patient conversations, EyeGPT should recognize potential urgent situations and guide the user toward appropriate professional/emergency care.

This should be a **safety workflow**, not an AI diagnosis.

55. Patient Data Ownership

The patient should have a clear view of:

- what information NEI stores,
- which doctors can access it,
- which clinics have access,
- what was shared,

- and how to request access/export where applicable.

This builds trust.

56. Patient Data Timeline

One of the best NEI features should ultimately be:

“My Eye Journey”

A single timeline showing:

Appointments



Examinations



Diagnoses



Prescriptions



Medications



Imaging



Procedures



Follow-ups

This turns fragmented medical data into something understandable.

57. Future XR Infrastructure

Do NOT build this into MVP.

Prepare an extension layer:

NEI Core



NEI XR SDK



└─ Eye Tracking

└─ Visual Field

└─ Amblyopia

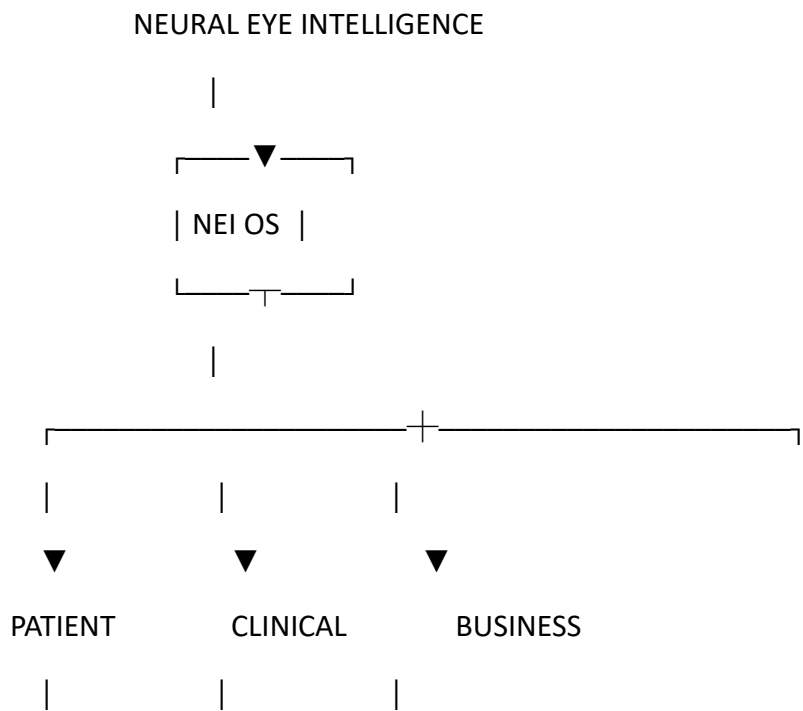
└─ Rehabilitation

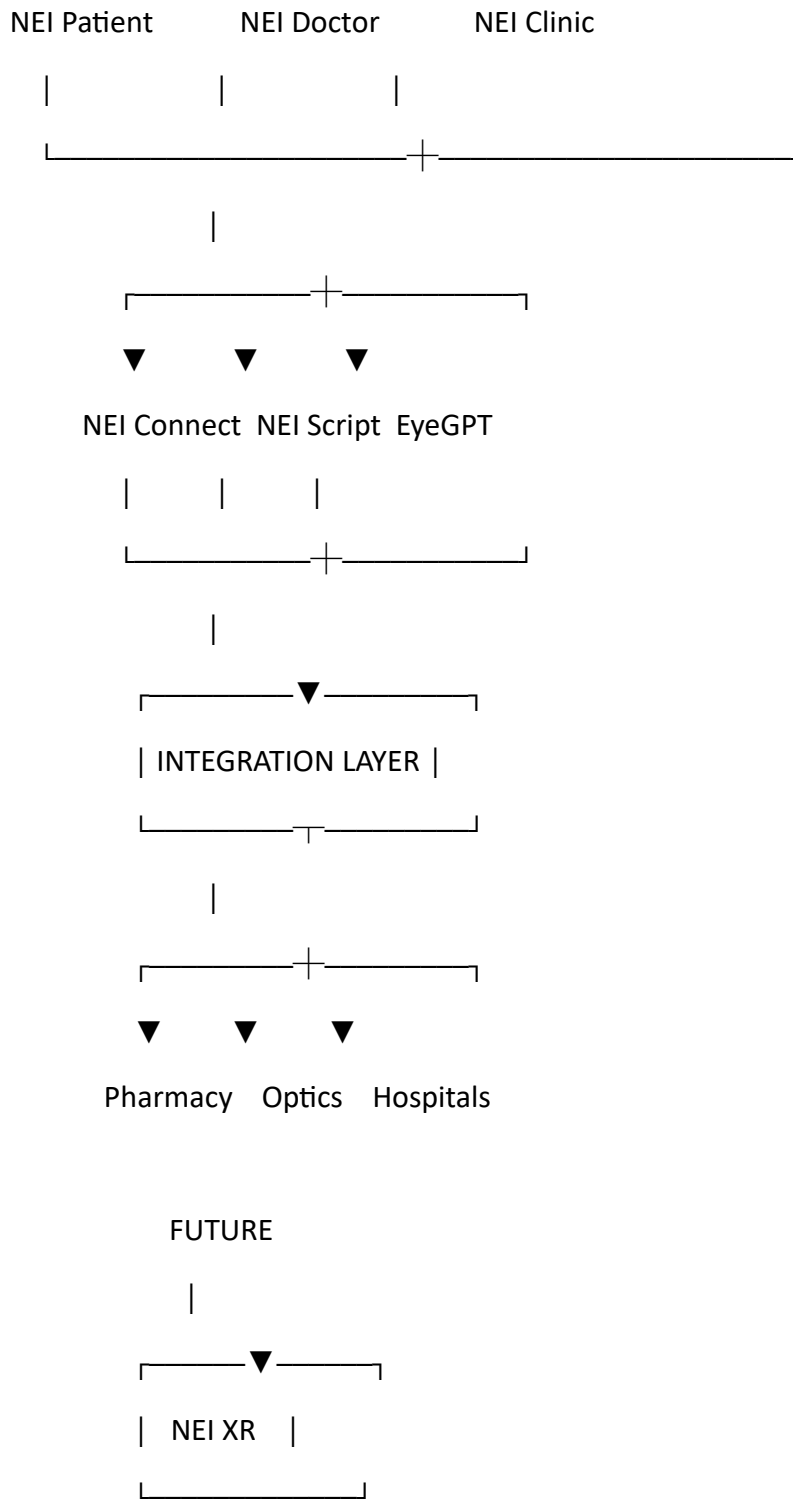
└─ Immersive Diagnostics

XR should communicate with NEI Core through secure APIs.

58. Recommended Product Hierarchy

The entire system can therefore be represented as:





59. Development Priority

This is where I would significantly change your original roadmap.

Phase 0 — Foundation

Build:

- architecture,
 - authentication,
 - organizations,
 - roles,
 - permissions,
 - database,
 - audit system,
 - API foundation,
 - design system.
-

Phase 1 — Clinical MVP

Patient

- sign up/sign in
- patient profile
- eye-care profile
- appointments
- prescriptions
- medical timeline
- notifications
- doctor messaging

Doctor

- sign in
- dashboard
- patient management

- appointment management
- clinical encounters
- eye examination
- prescriptions
- notes

Core

- RBAC
 - audit logs
 - document storage
 - notifications
-

Phase 2 — NEI Connect

Add:

- telemedicine,
 - waiting room,
 - chat,
 - file sharing,
 - consultation records,
 - automated summaries.
-

Phase 3 — EyeGPT

Add:

Patient

- education
- record explanation
- reminders

- appointment assistance

Doctor

- patient summaries
 - SOAP drafts
 - documentation assistance
 - clinical workflow assistance
-

Phase 4 — NEI Script + Pharmacy

Add:

- QR prescriptions,
 - pharmacy verification,
 - dispensing,
 - medication tracking,
 - refill workflows.
-

Phase 5 — NEI Optics

Add:

- optical shops,
- inventory,
- eyewear,
- prescription synchronization,
- orders.

Then:

- AI recommendations,
 - AR try-on.
-

Phase 6 — Advanced Intelligence

Add:

- predictive analytics,
 - imaging AI,
 - advanced clinical AI,
 - population insights.
-

Phase 7 — NEI XR

Finally:

- visual field,
 - eye tracking,
 - rehabilitation,
 - immersive diagnostics.
-

60. What the Developer AI Should NOT Do

This should be explicitly included in the development instructions.

The developer AI must **not**:

- build all modules simultaneously;
- treat future features as MVP requirements;
- create unnecessary microservices;
- allow AI to make autonomous medical decisions;
- expose patient data across organizations;
- hard-code external integrations;
- store medical files directly in PostgreSQL;
- bypass authorization for convenience;

- create generic healthcare workflows where ophthalmology-specific workflows are required.

61. Developer AI's Core Rules

The developer AI should follow these principles:

Rule 1

Security before convenience.

Rule 2

Patient privacy before functionality.

Rule 3

Doctor remains responsible for clinical decisions.

Rule 4

AI assists; AI does not replace clinicians.

Rule 5

Every module must have clear authorization boundaries.

Rule 6

Every important clinical action must be auditable.

Rule 7

Build modularly so future NEI products can reuse the same core.

Rule 8

Accessibility is a core requirement, not a later feature.

Rule 9

The patient experience must remain simple even when the backend is complex.

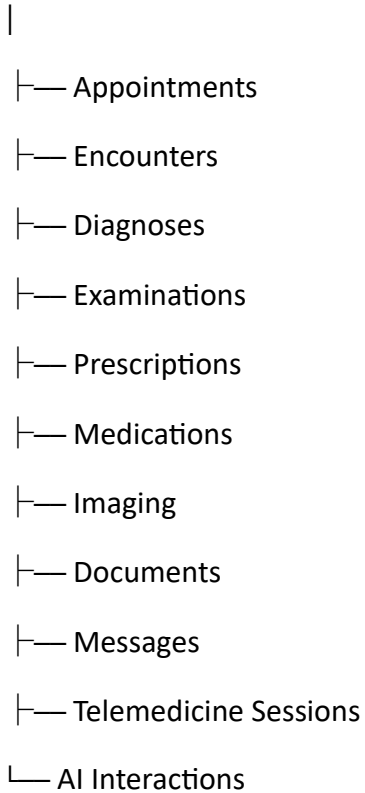
Rule 10

Do not sacrifice the MVP for futuristic functionality.

62. The Most Important Database Relationship

Everything should ultimately connect around:

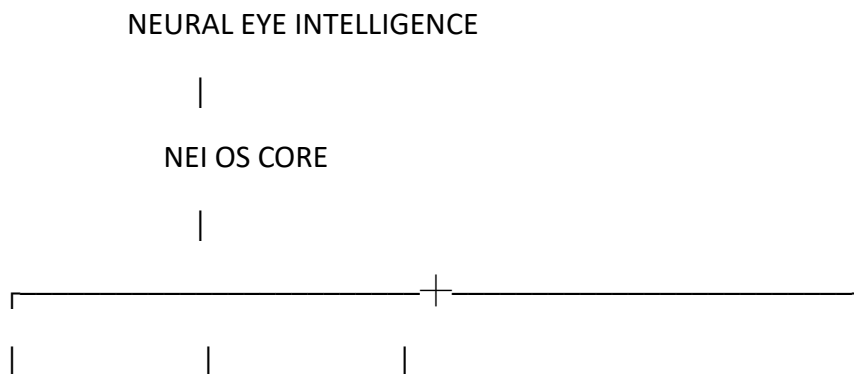
PATIENT

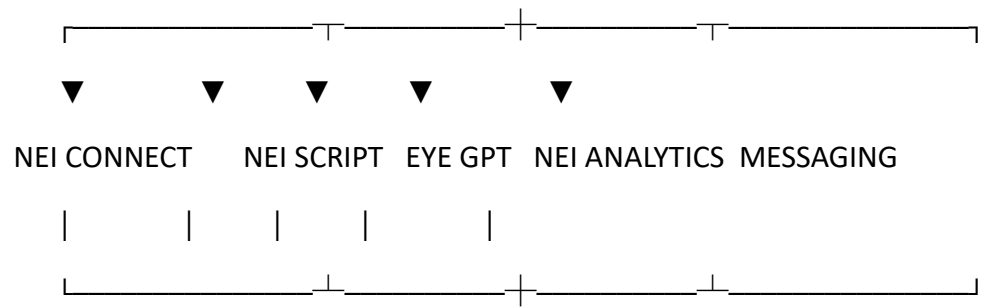
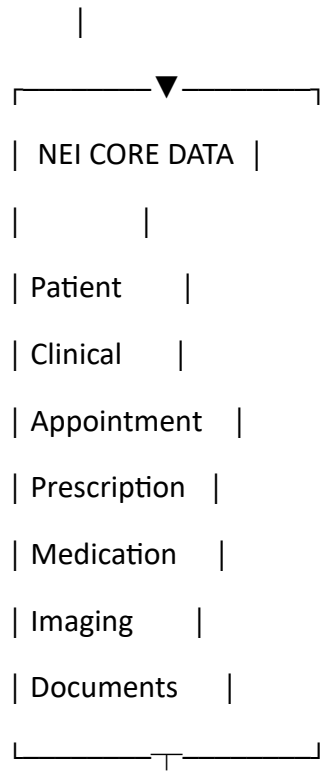
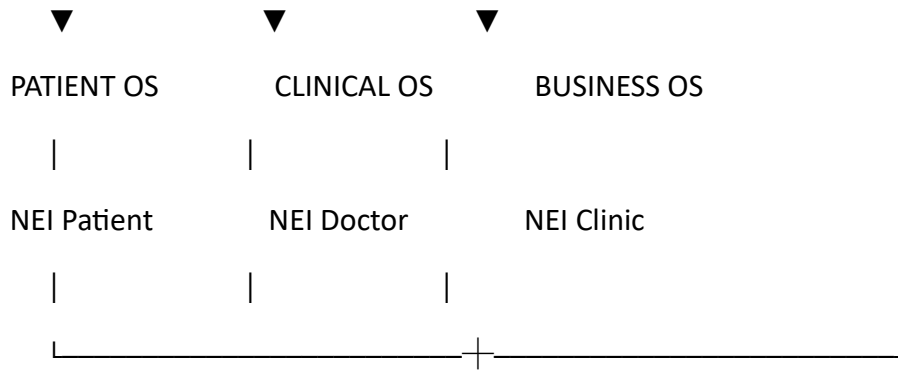


This becomes the **longitudinal patient eye-care record**.

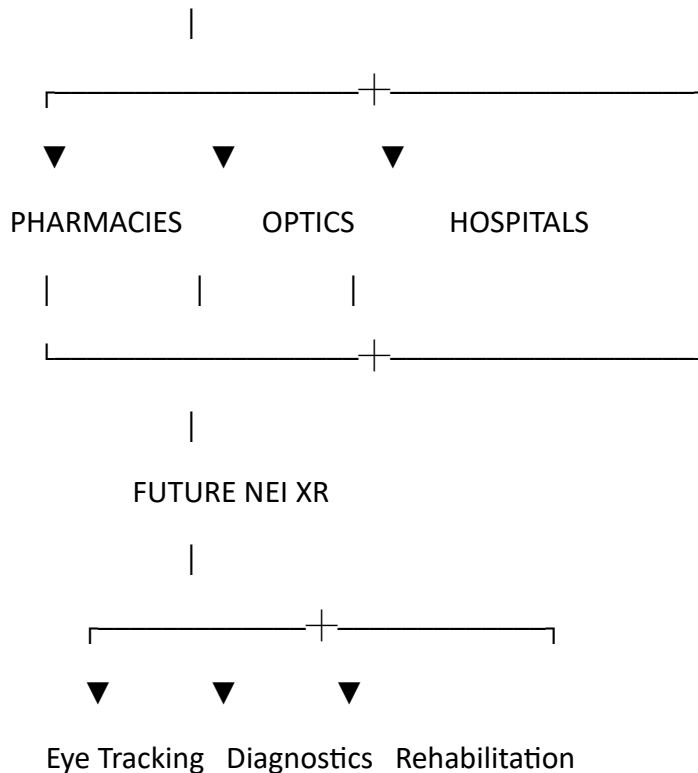
63. Final NEI Infrastructure Vision

The complete system should ultimately look like this:





INTEGRATION HUB



My recommendation

Give the original NEI document to a developer AI as-is. This infrastructure should become the foundation of a much larger **NEI Product Requirements + Technical Specification**, where every screen, user role, workflow, database entity, API, permission, and AI behavior is explicitly defined.

The next useful step is to turn this into a **complete developer-ready specification**, starting with the **Patient App screen-by-screen**, including **Sign Up → Onboarding → Patient Profile → Eye Health Record → Appointments → Doctor → Telemedicine → Prescriptions → EyeGPT → Settings**, followed by the Doctor and Admin systems.

Which should I build next: Patient App screen-by-screen, Doctor platform, or complete database/API specification?